DECOY HONEYPOT ALERT

SUBMITTED BY

- MOHAMED ANEES.M

CYBER SECURITY AND PYTHON

SARVA SUDARSANA ACADEMY



OCTOBER-2026

CONTENTS

1.SYNOPSIS

2.ABSTRACT

3.CONFIGRATION

4. IMAGE

5. CODE

6. FURTHER SCOPE OF PROJECT

1.SYNOPSIS

The objective of the project is titled as **"AUTOMATED DIRECTORY-WIDE FILE INTEGRITY MONITORING HONEYPOT"** is to develop a robust host-based intrusion detection system utilizing **Python** for real-time monitoring.

The core script operates in the background monitoring the target directory using standard libraries like **os**, **hashlib**, **socket**, and **getpass** for scalable and light-weight operations. The operating system used is **Windows 11 / Linux**.

The existing system relies on manual file verification or single-file decoy monitoring, limiting real-time security across the entire workspace. The Proposed system utilizes recursive directory hashing (**SHA-256**) for dynamic integrity verification and instant **Telegram REST API Webhooks**. This solution ensures immediate threat alerting and minimizes detection latency.

This project consists of four modules, they are: **Module 1 - Directory Hash Engine:** Dynamically scans all non-python files using recursive hashing to capture initial state baselines. **Module 2 - Real-Time Monitor Loop:** Periodically checks cryptographic checksums to detect file modifications or additions instantly. **Module 3 - Host Forensic Extractor:** Captures key contextual details including user profile context and host IP address. **Module 4 - Telegram Webhook Dispatcher:** Formats and transmits automated security alerts directly to the administrator's Telegram device.

2.ABSTRACT

The growing frequency of unauthorized system access and file tampering poses significant challenges to modern host-level security. Traditional intrusion detection mechanisms often require manual target file specification or rely on resource-heavy third-party frameworks.

This project presents a host-based, dynamic file integrity monitoring system designed to protect sensitive directory structures through automated cryptographic verification.

Implemented entirely in **Python**, the system utilizes **SHA-256 algorithm** hashing to construct baseline security states for all non-executable files within a designated workspace.

By executing a continuous background monitoring loop, the system dynamically compares real-time checksums against saved baseline states to detect file alterations, creations, or unauthorized modifications instantly.

Upon identifying a breach, an integrated forensic module extracts critical host metadata, including the active user context and local IP address.

The system then formats and dispatches a structured incident report via the **Telegram REST API Webhook**, delivering instant push notifications to security administrators.

Built without external library dependencies, this lightweight honeypot framework offers low latency, zero configuration overhead, and an efficient approach to automated endpoint surveillance and incident response.

3.CONFIGRATION

1.OPERATING SYSTEM:

WINDOWS-11

1.Hardware Configuration

System Type: Personal Computer

Memory: 4 GB RAM minimum

CPU: Dual-Core 2.0 GHz or higher processor

Storage: 20 GB available disk space (for system logs and file monitoring)

2.Software Configuration

Operating System: Windows 10/11

Programming Environment: Python 3.8+ Runtime Environment

Built-in Libraries: os, time, hashlib, getpass, socket, urllib.request, urllib.parse

Third-Party Services: Telegram Bot API (via BotFather Webhook Framework)

4.IMAGE

1.SOURCE CODE

```
import os
import time
import hashlib
import getpass
import socket
import urllib.request
import urllib.parse

FOLDER = "C:/Users/Jaitha/OneDrive/Documents"
BOT_TOKEN = "8804369878:AAEhV7_FYLDyTZtKiGZbFv9uIIrAyJ2l4_k"
CHAT_ID = "7600936513"

def get_hash(filepath):
    try:
        with open(filepath, "rb") as f:
            return hashlib.sha256(f.read()).hexdigest()
    except Exception:
        return None

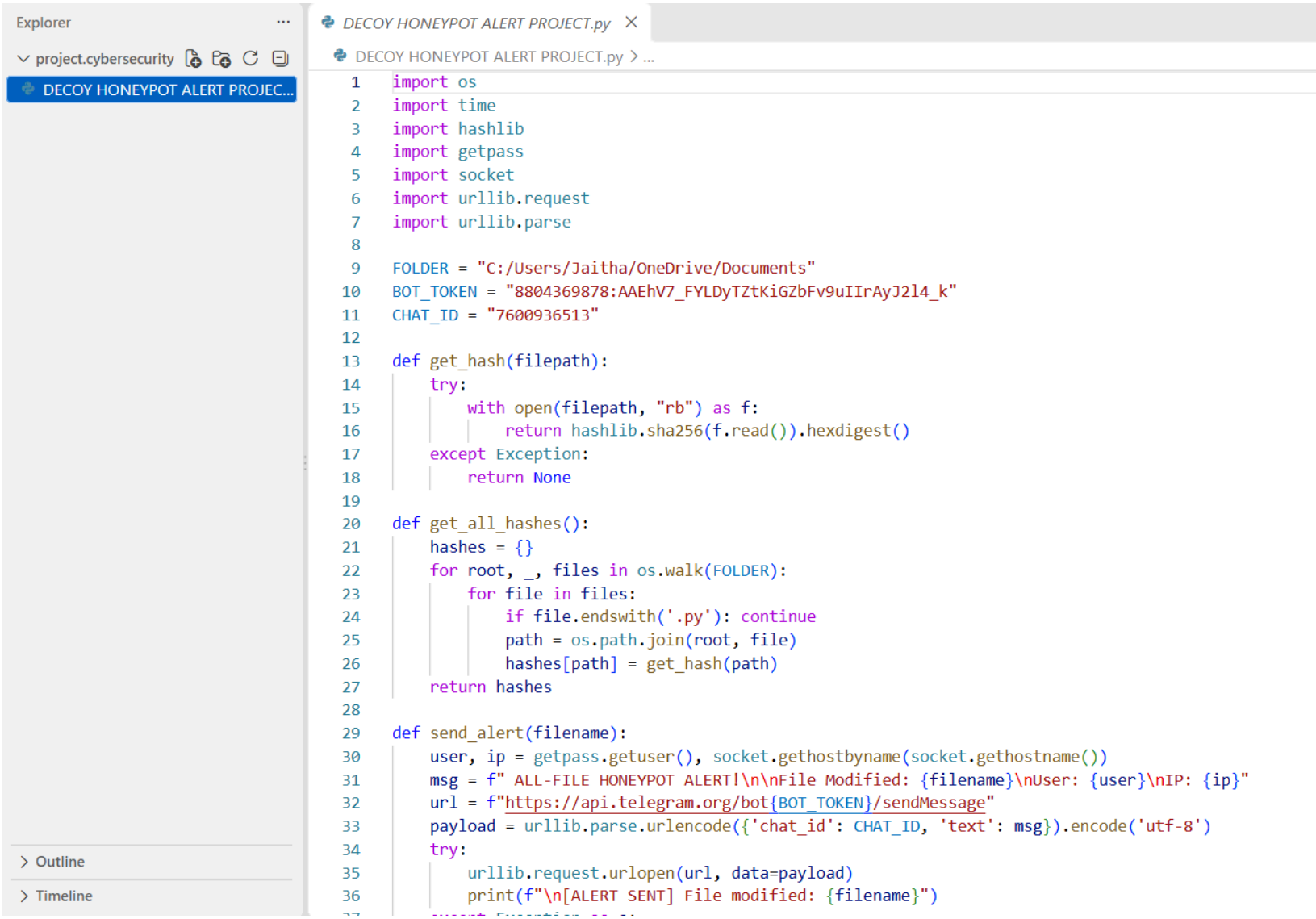
def get_all_hashes():
    hashes = {}
    for root, _, files in os.walk(FOLDER):
        for file in files:
            if file.endswith('.py'): continue
            path = os.path.join(root, file)
            hashes[path] = get_hash(path)
    return hashes

def send_alert(filename):
    user, ip = getpass.getuser(), socket.gethostbyname(socket.gethostname())
    msg = f" ALL-FILE HONEYPOT ALERT!\n\nFile Modified: {filename}\nUser: {user}\nIP: {ip}"
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
    payload = urllib.parse.urlencode({'chat_id': CHAT_ID, 'text': msg}).encode('utf-8')
    try:
        urllib.request.urlopen(url, data=payload)
        print(f"\n[ALERT SENT] File modified: {filename}")
    except Exception as e:
        print(f"\n[ERROR] Telegram send failed: {e}")

known_files = get_all_hashes()
print("--- ALL FILES HONEYPOT ACTIVE ---")

while True:
    current_files = get_all_hashes()
    for path, h in current_files.items():
        if path in known_files and h != known_files[path]:
            send_alert(path)
```

2. VS OPEN


Explorer
project.cybersecurity
DECOY HONEYPOT ALERT PROJEC...
Outline
Timeline
DECOY HONEYPOT ALERT PROJECT.py
DECOY HONEYPOT ALERT PROJECT.py > ...
1 import os
2 import time
3 import hashlib
4 import getpass
5 import socket
6 import urllib.request
7 import urllib.parse
8
9 FOLDER = "C:/Users/Jaitha/OneDrive/Documents"
10 BOT_TOKEN = "8804369878:AAEhV7_FYLDyTZtKiGZbFv9uIIrAyJ2l4_k"
11 CHAT_ID = "7600936513"
12
13 def get_hash(filepath):
14 try:
15 with open(filepath, "rb") as f:
16 return hashlib.sha256(f.read()).hexdigest()
17 except Exception:
18 return None
19
20 def get_all_hashes():
21 hashes = {}
22 for root, _, files in os.walk(FOLDER):
23 for file in files:
24 if file.endswith('.py'): continue
25 path = os.path.join(root, file)
26 hashes[path] = get_hash(path)
27 return hashes
28
29 def send_alert(filename):
30 user, ip = getpass.getuser(), socket.gethostbyname(socket.gethostname())
31 msg = f" ALL-FILE HONEYPOT ALERT!\n\nFile Modified: {filename}\nUser: {user}\nIP: {ip}"
32 url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
33 payload = urllib.parse.urlencode({'chat_id': CHAT_ID, 'text': msg}).encode('utf-8')
34 try:
35 urllib.request.urlopen(url, data=payload)
36 print(f"\n[ALERT SENT] File modified: {filename}")

3.TERMINAL OUTPUT

```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

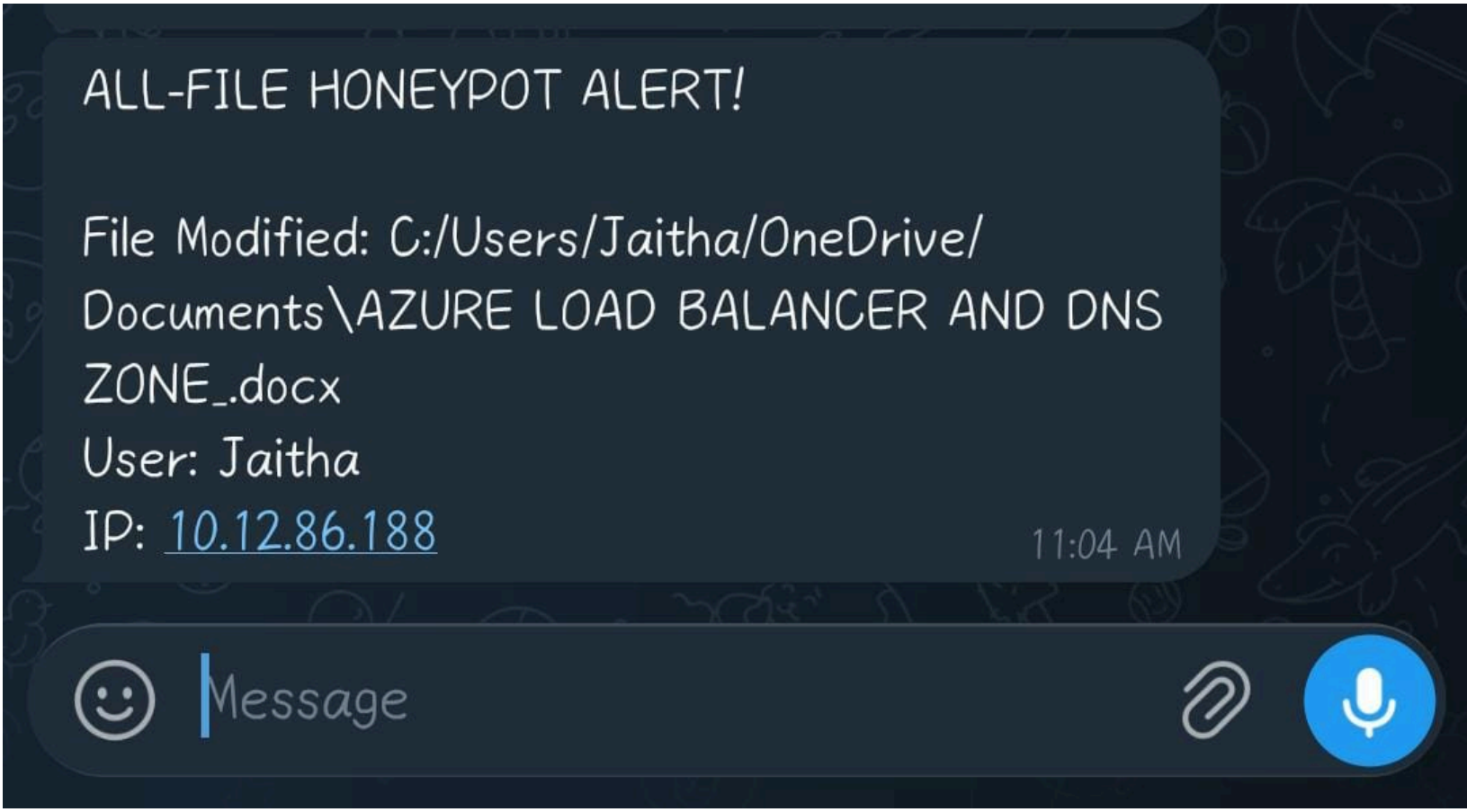
C:\Users\Jaitha\project.cybersecurity>code .

C:\Users\Jaitha\project.cybersecurity>python "DECOY HONEYPOT ALERT PROJECT.py"
--- ALL FILES HONEYPOT ACTIVE ---
[INFO] Monitoring all files...
[ALERT SENT] File modified: C:/Users/Jaitha/OneDrive/Documents\i.log
[INFO] Monitoring all files...
[ALERT SENT] File modified: C:/Users/Jaitha/OneDrive/Documents\AZURE LOAD BALANCER AND DNS ZONE_.docx
[INFO] Monitoring all files...
```

4.TXT.FILE CHANGES

```
13:32:45.290 00002564 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:32:51.421 00004188 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:32:54.244 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:32:57.531 00000144 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:03.625 000014DC - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:09.795 00001FFC - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:14.280 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:33:16.024 00001490 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:22.165 00002028 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:28.262 00004310 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:34.347 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:33:34.457 00001FF4 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:40.576 00001F8C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:46.811 0000340C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:52.969 0000299C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:33:54.368 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:33:59.153 00001914 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:05.365 00000C78 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:11.567 00000BD4 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:14.395 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:34:17.728 00004A18 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:23.809 0000013C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:30.044 00001B38 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:34.411 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:34:36.161 00000C68 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:42.352 00000FBC - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:48.540 000028F0 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:34:54.418 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:34:54.745 00002544 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:00.852 00003F0C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:07.056 00002F18 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:13.255 00000C2C - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|NBHBCCHs
13:35:14.464 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
13:35:19.371 00000828 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:25.477 00001940 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:31.659 00002938 - ERROR: Couldn't connect to routers 136.243.104.242:443,80,8080|136.243.104.235:443,80,8080|136.243.18.81:443,80,8080|
13:35:34.491 00003498 - ERROR: Error {12007} occured while connecting to server "http://rl.ammyy.com"
```

4. TELEGRAM ALERT



4.CODE

SOURCE CODE:

```
import os
import time
import hashlib
import getpass
import socket
import urllib.request
import urllib.parse

FOLDER = "C:/Users/Jaitha/OneDrive/Documents"
BOT_TOKEN = "8804369878:AAEhV7_FYLDyTZtKiGZbFv9uIIrAyJ2l4_k"
CHAT_ID = "7600936513"

def get_hash(filepath):
    try:
        with open(filepath, "rb") as f:
            return hashlib.sha256(f.read()).hexdigest()
    except Exception:
        return None

def get_all_hashes():
    hashes = {}
    for root, _, files in os.walk(FOLDER):
        for file in files:
            if file.endswith('.py'): continue
            path = os.path.join(root, file)
            hashes[path] = get_hash(path)
    return hashes

def send_alert(filename):
    user, ip = getpass.getuser(), socket.gethostbyname(socket.gethostname())
    msg = f" ALL-FILE HONEYPOT ALERT!\n\nFile Modified: {filename}\nUser:
{user}\nIP: {ip}"
    url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"
```

```
    payload = urllib.parse.urlencode({'chat_id': CHAT_ID, 'text':
msg}).encode('utf-8')
    try:
        urllib.request.urlopen(url, data=payload)
        print(f"\n[ALERT SENT] File modified: {filename}")
    except Exception as e:
        print(f"\n[ERROR] Telegram send failed: {e}")

known_files = get_all_hashes()
print("--- ALL FILES HONEYPOT ACTIVE ---")

while True:
    current_files = get_all_hashes()
    for path, h in current_files.items():
        if path in known_files and h != known_files[path]:
            send_alert(path)
    known_files = current_files
    print("[INFO] Monitoring all files...", end="\r")
    time.sleep(2)
```

4. FURTHER SCOPE OF PROJECT

The scope of this project focuses on developing and deploying a host-based intrusion detection system designed for real-time monitoring and threat alerting across local file systems.

1. Functional Scope

Dynamic Directory Monitoring: Automatically scans and tracks all existing and newly created non-executable files within a designated folder structure without needing hardcoded filenames.

Cryptographic Tamper Detection: Utilizes the **SHA-256 cryptographic algorithm** to detect file contents modification, corruption, or unauthorized edits with zero false positives from timestamp manipulation.

Forensic Metadata Extraction: Captures critical endpoint context during a security breach, including the active local **Username** and current host **IP Address**.

Real-time Incident Notification: Integrates directly with the **Telegram REST API Webhook** framework to send instant security alerts directly to the administrator's mobile/desktop device.

2. Technical & Architectural Scope

Zero External Dependencies: Built using standard Python modules (`os`, `hashlib`, `socket`, `getpass`, `urllib`), ensuring cross-platform deployment without installing external `pip` packages.

Low System Overhead: Operates continuously with minimal CPU and memory consumption, making it suitable for endpoint protection and background auditing.

3. Future Enhancements (Out of Scope for Current Phase)

Automated threat response mechanisms (e.g., blocking suspicious processes or IP addresses upon detection).

Graphical User Interface (GUI) for visually setting up target paths and viewing historical alert logs.

Cloud database integration for centralized multi-device log management.